QORVO SCHOLARSHIP AND TRAINING PROGRAM



SPECTIFICATION

Project: hms_timer

Author: Pham Truong Giang
Instructor: Nguyen Huu Quang

Ha Noi, 09/2026

MỤC LỤC

1. Design Overview....1
1.1. Tổng quan chức năng....1
1.2. Block Diagram....2
1.3. I/O Table....2
2. Architecture....3
2.1. Detail block diagram....3
2.2. Instance Table....3
3. Detailed Block Description....5
3.1. Block: reset_sync (Reset Synchronizer)....5
3.1.1. Tổng quan....5
3.1.2. Table....6
3.1.3. Logic hoạt động chi tiết....6
3.2. Block: up_down_cond (Input Button Up/Down Conditioning)....8
3.2.1. Tổng quan....8
3.2.2. I/O Table....8
3.2.3. Logic hoạt động chi tiết....9
3.3. Block: sel_cond (Select Button Conditioning)....13
3.3.1. Tổng quan....13
3.3.2. I/O Table....13
3.3.1. Logic hoạt động chi tiết....14
3.4. Block: sel_fsm (Select Field FSM)....15
3.4.1. Tổng quan....15
3.4.2. I/O Table....16
3.4.3. Logic hoạt động chi tiết....17
3.5. Block: freq_divider (Frequency Divider)....23
3.5.1. Tổng quan....23
3.5.2. I/O Table....24

3.5.3. Logic hoạt động chi tiết....24
3.6. Block: main_counter (Main Time Counter) 26
3.6.1. Tổng quan....26
3.6.2. I/O Table....27
3.6.3. Logic hoạt động chi tiết....28
3.7. Block: adjust_counter (Adjust Counter) 31
3.7.1. Tổng quan....31
3.7.2. I/O Table....32
3.7.3. Logic hoạt động chi tiết....33
3.8. Block: control_mux (Control and Output MUX)....38
3.8.1. Tổng quan....38
3.8.2. I/O Table....39
3.8.3. Logic hoạt động chi tiết....40

DANH MỤC HÌNH VẼ

Hình 1.1: Block Diagram module hms_timer (khối top level).....2
Hình 2.1: Tổng quan kiến trúc tổng quan cho module hms_timer.....3
Hình 3.1: Block diagram module rstn_sync 5
Hình 3.2: Block diagram module up_down_cond.....8
Hình 3.3: Mô tả chi tiết logic hoạt động của module up_down_cond.....9
Hình 3.4: Block diagram module sel_cond 13
Hình 3.5: Detail block diagram module sel_cond 14
Hình 3.6: Block diagram modue sel_fsm.....15
Hình 3.7: State diagram cho hoạt động của module sel_fms.....19
Hình 3.8: Block diagram module freq_divider 23
Hình 3.9: Block diagram module mmain_counter 26
Hình 3.10: Block diagram module adjust_counter.....31
Hình 3.11: Detail logic of FSM decoder.....34
Hình 3.12: Block diagram module control_mux.....38

DANH MỤC BẢNG BIỂU

Bảng 1.1: Tổng quan tín hiệu input và output khối hms_timer....2
Bảng 2.1: Tên instance ứng với các module trong project 3
Bảng 3.1: I/O table module reset_sync 6
Bảng 3.2: Internal signals table module reset_sync....6
Bảng 3.3: Hành vi của tín hiệu rstn khi assert 7
Bảng 3.4: Hành vi của tín hiệu rstn khi deassert....7
Bảng 3.5: I/O table module up_down_cond....8
Bảng 3.6: Parameter of module up_down_cond 9
Bảng 3.7: Internal signals tầng FF synchronizer module up_down_cond....10
Bảng 3.8: Hành vi signals tầng FF synchronizer module up_down_cond....10
Bảng 3.9: Internals signals tầng debounce module up_down_cond 10
Bảng 3.10: Hành vi tầng debounce module up_down_cond....11
Bảng 3.11: Internal signals tầng rising edge detector module up_down_cond 11
Bảng 3.12: Hành vi tầng rising edge detector module up_down_cond....12
Bảng 3.13: Parameter of module sel_cond....13
Bảng 3.14: I/O table module sel_cond....14
Bảng 3.15: Paramenter of module sel_fsm....16
Bảng 3.16: I/O table module sel_fsm 16
Bảng 3.17: State encoding 17
Bảng 3.18: Internal signals of the module sel_fsm....18
Bảng 3.19: Share function logic counter....18
Bảng 3.20: Điều kiện ưu tiên và hành vi tại state PL_NORMAL 19
Bảng 3.21: Điều kiện ưu tiên và hành vi tại state PL_SEL_MIN....20
Bảng 3.22: Điều kiện ưu tiên và hành vi tại state PL_SEL_HOUR 21
Bảng 3.23: Điều kiện ưu tiên và hành vi tại state PL_SEL_SEC 22
Bảng 3.24: Case default và điều kiện ngoài clk_en_10khz 23
Bảng 3.25: I/O table module freq_divider....24
Bảng 3.26: Internal signals of the module freq_divider 24
Bảng 3.27: Mức độ ưu tiên giữa các điều kiện và hành vi tương ứng của logic clk_en_10khz generation 25
Bảng 3.28: Mức độ ưu tiên giữa các điều kiện và hành vi tương ứng của logic clk_en_10khz generation 25
Bảng 3.29: I/O table module time_counter 27

Bảng 3.30: Internal signals of the module time_counter.....28
Bảng 3.31: Load data bit mapping 28
Bảng 3.32: Mức độ ưu tiên điều kiện và hành vi tương ứng của logic update sec register 28
Bảng 3.33: Mức độ ưu tiên điều kiện và hành vi tương ứng của logic update min register 29
Bảng 3.34: Mức độ ưu tiên điều kiện và hành vi tương ứng của logic update hour register 30
Bảng 3.35: I/O table of the module adjust_counter 32
Bảng 3.36: Internal signals of the module adjust_counter.....33
Bảng 3.37: State encoding for FSM decoder.....33
Bảng 3.38: Logic cập nhật giá trị action 34
Bảng 3.39: Route action sang trường tương ứng theo fsm_state.....34
Bảng 3.40: Mức độ ưu tiên của điều kiện và hành vi tương ứng của cập nhật logic cho sec register.....35
Bảng 3.41: Mức độ ưu tiên của điều kiện và hành vi tương ứng của cập nhật logic cho min register 36
Bảng 3.42: Mức độ ưu tiên của điều kiện và hành vi tương ứng của cập nhật logic cho hour register 36
Bảng 3.43: I/O table of the control_mux module.....39
Bảng 3.44: Internal signal of the control_mux module.....40
Bảng 3.45: Localparam of the control_mux module 41
Bảng 3.46: Mức độ ưu tiên về điều kiện và hành vi tương ứng của fsm_sate_d register .41
Bảng 3.47: Logic phát tín hiệu load_shd_en và đích đến của nhóm bit load_shd_data ...42
Bảng 3.48: Logic phát load_en và đích đến của nhóm bit load_data vào main khi confirm 42
Bảng 3.49: Logic tín hiệu clear_shd.....43
Bảng 3.50: MUX chọn hiển thị.....43

1. Design Overview

1.1. Tổng quan chức năng

Module **hms_timer** là bộ đếm thời gian thực hiển thị giờ - phút - giây, hoạt động ở tần số clock 1 MHz với reset bất đồng bộ tích cực mức thấp.

Module thực hiện hai chức năng chính:

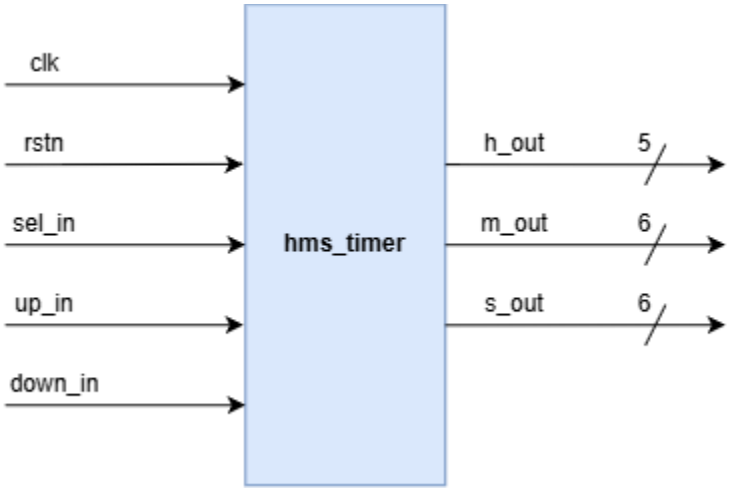
Chế độ NORMAL (Free-run counting): Đồng hồ tự động đếm giây, phút, giờ theo thời gian thực. Mỗi giây, giá trị second tăng 1. Khi second đạt 59, nó quay về 0 và nhớ lên giá trị minute. Tương tự, minute nhớ lên hour. Hour wrap tại 23 về 0 (định dạng 24h). Các nút Up và Down không có tác dụng trong chế độ này.

Chế độ chỉnh giờ (Time adjustment): Người dùng giữ nút Select liên tục 3 giây để vào chế độ chỉnh. Khi vào chế độ chỉnh, đồng hồ hiển thị chế độ điều chỉnh. Thứ tự chỉnh các trường như sau: SEL_MIN → SEL_HOUR → SEL_SEC. Tại mỗi trường, nút Up tăng giá trị thêm 1, nút Down giảm giá trị đi 1. Khi Up và Down cùng xuất hiện trong 1 chu kỳ clock, Up được ưu tiên. Giá trị chỉ wrap trong trường đang chỉnh, không nhớ sang trường khác (ví dụ: tăng minute từ 59 sẽ wrap về 0 mà hour giữ nguyên).

Nhấn nút Select sẽ chuyển sang trường tiếp theo. Giá trị các trường chỉ được ghi nhận vào đồng hồ khi hoàn thành toàn bộ quy trình chỉnh (SEL_MIN → SEL_HOUR → SEL_SEC → NORMAL). Tại trường cuối cùng (SEL_SEC), nhấn nút Select sẽ xác nhận toàn bộ giá trị đã chỉnh và quay về chế độ NORMAL, đồng hồ đếm tiếp từ giá trị mới.

Nếu tại bất kỳ trạng thái chỉnh nào mà không có thao tác trong 5 giây (timeout), đồng hồ quay về chế độ NORMAL. Toàn bộ giá trị đã chỉnh tại tất cả các trường bị hủy bỏ, đồng hồ đếm tiếp từ giá trị giá trị trước khi vào chế độ chỉnh.

1.2. Block Diagram



Hình 1.1: Block Diagram module hms_timer (khối top level)

1.3. I/O Table

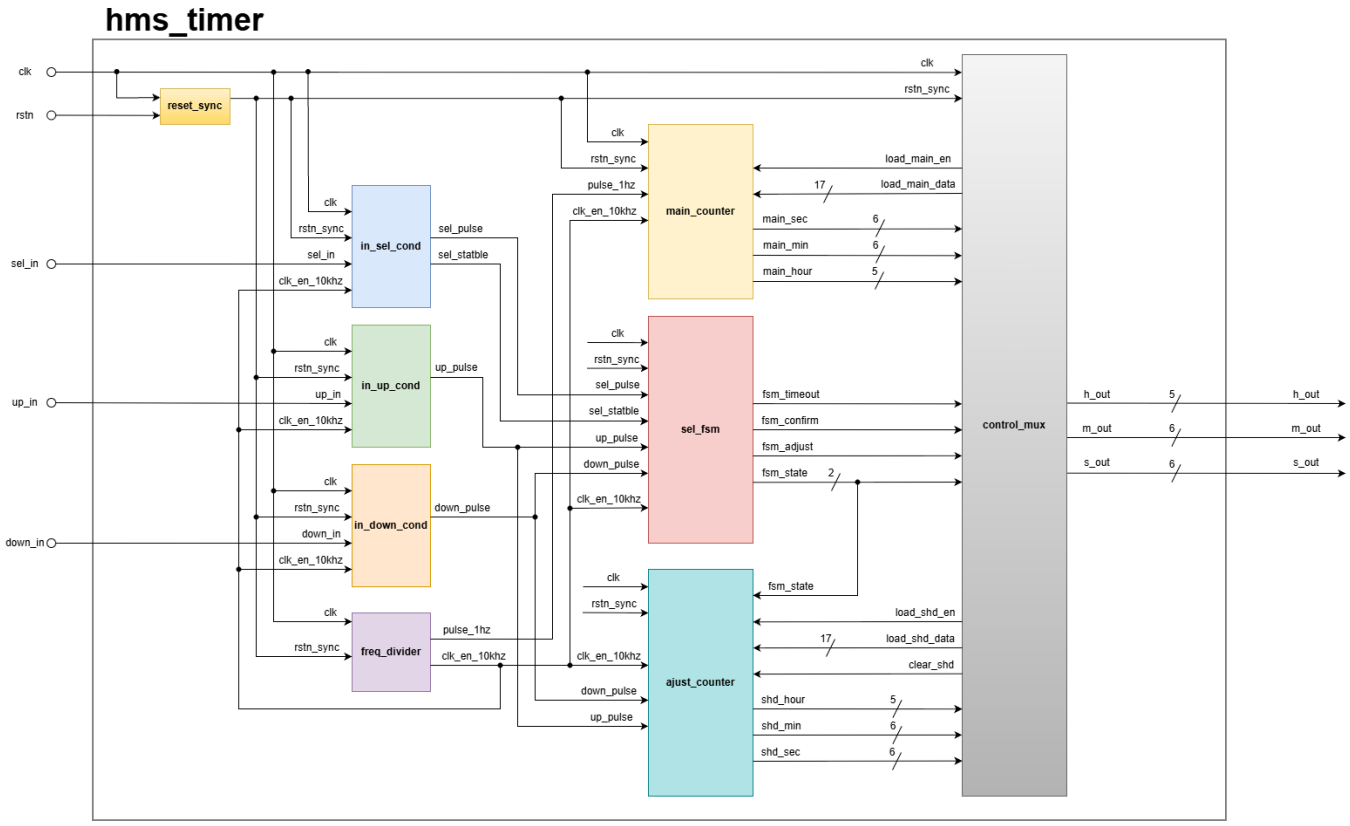
Bảng 1.1: Tổng quan tín hiệu input và output khối hms_timer

Port	Chiều	Width	Mô tả
clk	Input	1	Tín hiệu clock hệ thống, tần số 1 MHz.
rstn	Input	1	Tín hiệu reset bất đồng bộ, tích cực mức thấp.
sel_in	Input	1	Tín hiệu chọn trường thời gian. Mỗi cạnh lên chuyển trường đang chọn: MIN → HOUR → SEC → NORMAL.
up_in	Input	1	Tín hiệu tăng giá trị. Mỗi cạnh lên tăng giá trị trường đang chọn thêm 1.
down_in	Input	1	Tín hiệu giảm giá trị. Mỗi cạnh lên giảm giá trị trường đang chọn đi 1.
h_out	Output	5	Giá trị giờ hiện tại, phạm vi 0–23.
m_out	Output	6	Giá trị phút hiện tại, phạm vi 0–59.
s_out	Output	6	Giá trị giây hiện tại, phạm vi 0–59.

2. Architecture

2.1. Detail block diagram

Module hms_timer gồm 8 sub-module, được instantiate thành 9 instance.



Hình 2.1: Tổng quan kiến trúc tổng quan cho module hms_timer

2.2. Instance Table

Bảng 2.1: Tên instance ứng với các module trong project

Instance Name	Module Name	Mô tả
u_reset_sync	reset_sync	Đồng bộ hóa rstn vào clock domain (Async Assert / Sync Deassert)
u_sel_cond	sel_cond	Xử lý nút Select: đồng bộ, debounce, tạo sel_pulse và sel_stable

u_up_cond	up_down_cond	Xử lý nút Up: đồng bộ, debounce, tạo up_pulse
u_down_cond	up_down_cond	Xử lý nút Down: đồng bộ, debounce, tạo down_pulse
u_sel_fsm	sel_fsm	FSM 4-state quản lý chế độ hoạt động, long press và timeout detect
u_freq_divider	freq_divider	Chia tần 1 MHz xuống clk_en_10khz rồi xuống pulse_1hz
u_main_counter	main_counter	Đếm thời gian chính (free-run), carry chain từ sec sang min sang hour
u_adjust_counter	adjust_counter	Shadow register phục vụ chế độ chỉnh, wrap trong từng trường
u_control_mux	control_mux	Điều phối load/clear giữa main và adjust, MUX hiển thị ra ngoài

Tổng quan về hoạt động của 9 sub-module:

Module **up_down_cond** (input button up/down conditioning) là khối xử lý tín hiệu đầu vào cho cả 3 nút, tương ứng với 3 instance mà ***Bảng 2.1*** đề cập. Tín hiệu nút bấm bản chất là bất đồng bộ với clock và bị rung cơ học (bouncing) hay nhiễu khi nhấn/nhả. Khối này giải quyết cả hai vấn đề thông qua pipeline 3 tầng:

- *Debounce Counter*: Chỉ công nhận sự thay đổi input khi nó giữ ổn định liên tục một thời gian N nhất định, loại bỏ các cạnh giả do rung động cơ học hoặc nhiễu. Trong thiết kế là 10 ms.
- *2-FF Synchronizer*: Đưa tín hiệu bất đồng bộ vào clock domain, giảm xác suất metastability.
- *Rising Edge Detector*: Chuyển tín hiệu dạng level thành 1 pulse đúng 1 chu kỳ clock tại thời điểm rising edge.

Module **sel_cond** về logic hoạt động và hành vi giống với module **up_down_cond** nhưng khác một điểm duy nhất là ngoài hoạt động đã liệt kê, khối còn đưa ra output xử lý việc giữ nút in_sel.

Module **sel_fsm** (Select FSM) là máy trạng thái quản lý trường thời gian đang được chọn để chỉnh. Mỗi khi nhận được sel_pulse, FSM chuyển sang trường tiếp theo.

Module **freq_divider** (Clock Divider): Bộ chia tần số, chuyển clock 1 MHz thành pulse clk_en_10khz với tần số 10 kHz và pulse_1hz với tần số 1 Hz.

Module **main_counter** là khối xử lý việc đếm thời gian, khối này luôn đếm và cập nhật thời gian khi user muốn chỉnh giờ chạy.

Moduel **adjust_counter** là khối xử lý logic user muốn chỉnh giờ, các tín hiệu từ Top vào (sel_in, up_in, down_in) sẽ tác động vào khối để chỉnh giờ hiển thị và cập nhật trở lại **main_counter.**

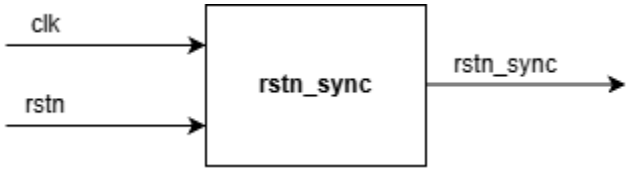
Module **rstn_sync** là khối đồng bộ hóa tín hiệu reset (rstn) tránh metastability khi tín hiệu rstn chuyển từ 0 lên 1. Khối đưa ra output rstn_sync và đi đến tất cả các internal module phía sau.

Module **control_mux** là khối điều phối hoạt động của logic hiển thị và cập nhật dữ liệu của user khi vào chế độ chỉnh giờ.

3. Detailed Block Description

3.1. Block: reset_sync (Reset Synchronizer)

3.1.1. Tổng quan



Hình 3.1: Block diagram module rstn_sync

reset_sync là khối đồng bộ hóa tín hiệu reset từ ngoài vào clock domain của hệ thống. Tín hiệu rstn đi vào module **hms_timer** bất đồng bộ với clock. Nếu đưa trực tiếp rstn vào async reset pin của toàn bộ các flip flop trong hệ thống, cạnh deassert (rstn: 0 →

1) có thể xảy ra gần cạnh lên clock gây metastability và khiến các flip flop thoát reset ở các thời điểm khác nhau, dẫn đến trạng thái khởi tạo không nhất quán.

Khối giải quyết bằng kỹ thuật Async Assert, Sync Deassert**:**

Assert (rstn: 1 → 0): bỏ qua clock, cả 2 FF reset ngay lập tức → rstn_sync xuống 0 không trễ.

Deassert (rstn: 0 → 1): tín hiệu đi qua 2 FF nối tiếp đồng bộ với clock → rstn_sync lên 1 sau 2 chu kỳ clock, đảm bảo metastability không lan truyền sang hệ thống.

Khối được sử dụng 1 lần trong hms_timer với instance name: u_reset_sync. Output rstn_sync được phân phối thay cho rstn tới tất cả các khối con khác.

3.1.2. Table

Bảng 3.1: I/O table module reset_sync

Port	Chiều	Width	Kết nối	Mô tả
clk	In	1	Từ top	Clock hệ thống 1 MHz
rstn	In	1	Từ top	Reset bất đồng bộ từ nguồn analog, tích cực mức thấp
rstn_sync	Out	1	Từ tất cả khối con khác	Reset đã đồng bộ hóa với clock domain, tích cực mức thấp

3.1.3. Logic hoạt động chi tiết

Internal signals:

Bảng 3.2: Internal signals table module reset_sync

Signal	Width	Dạng	Mô tả
meta_rstn	1	reg	Flip-flop tầng 1, có thể bị metastable tại cạnh deassert của rstn
inter_rstn_sync	1	reg	Flip-flop tầng 2 là giá trị đã ổn định sau khi FF1 có 1 chu kỳ clock để resolve.

Khối gồm 2 FF nối tiếp với 2 cơ chế reset khác nhau cho 2 hướng chuyển trạng thái:

Hành vi khi Assert:

Bảng 3.3: Hành vi của tín hiệu rstn khi assert

Điều kiện	**Hành vi**
rstn = 0	meta_rstn <= 0, inter_rstn_sync <= 0 (async reset, không chờ clock)

Cả 2 FF cùng được reset qua chân async reset ngay khi rstn xuống 0, bất kể vị trí của cạnh clock. rstn_sync xuống 0 ngay lập tức, đảm bảo toàn bộ hệ thống vào trạng thái reset đồng thời.

Hành vi khi Deassert:

Bảng 3.4: Hành vi của tín hiệu rstn khi deassert

Điều kiện	**Hành vi**
rstn = 1, posedge clk thứ 1	meta_rstn <= 1 (có thể metastable nếu cạnh rstn gần cạnh clk), inter_rstn_sync <= 0 (vẫn giữ)
rstn = 1, posedge clk thứ 2	meta_rstn = 1 (đã ổn định), inter_rstn_sync <= meta_rstn (= 1)

Khi rstn lên 1, FF1 (meta_rstn) sample input cố định D = 1. Nếu cạnh deassert quá gần cạnh clock, meta_rstn có thể rơi vào metastable trong 1 chu kỳ, nhưng FF2 (inter_rstn_sync) có gần 1 chu kỳ clock để meta_rstn ổn định trước khi FF2 sample.

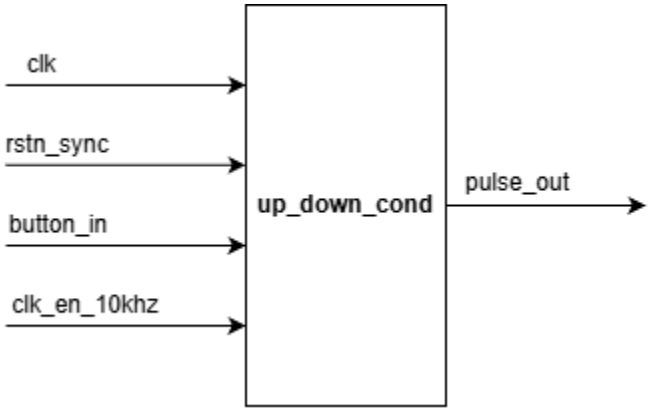
rstn_sync lên 1 sau 2 chu kỳ clock kể từ khi rstn được nhả.

Output

Output rstn_sync là output đi đến tất các cả các internal module của hms_timer. Output này chính là giá trị của inter_rstn_sync (rstn_sync = inter_rstn_sync).

3.2. Block: up_down_cond (Input Button Up/Down Conditioning)

3.2.1. Tổng quan



Hình 3.2: Block diagram module up_down_cond

up_down_cond là khối xử lý tín hiệu đầu vào từ nút bấm Up và Down. Tín hiệu nút bấm thô có 2 vấn đề: bất đồng bộ với clock domain gây metastability, và rung cơ học (bouncing) khi nhấn/nhả tạo nhiều cạnh giả.

Khối giải quyết cả 2 vấn đề và chuyển tín hiệu dạng level thành 1 pulse duy nhất mỗi lần nhấn thông qua pipeline 3 tầng nối tiếp: i) Tầng 1 — 2-FF Synchronizer: đưa tín hiệu bất đồng bộ vào clock domain, ii) Tầng 2 — Debounce Counter: lọc rung bằng cách yêu cầu input giữ ổn định liên tục một khoảng thời gian xác định. ii) Tầng 3 — Rising Edge Detector: tạo pulse 1 cycle tại rising edge của tín hiệu đã lọc.

Khối được sử dụng cho 2 instance: u_up_cond và u_down_cond. Cả 2 instance có hành vi giống hệt nhau, chỉ khác kết nối I/O.

3.2.2. I/O Table

Port:

Bảng 3.5: I/O table module up_down_cond

Port	Chiều	Width	Mô tả
clk	Input	1	Clock hệ thống 1 MHz
rstn_sync	Input	1	Reset bất đồng bộ và đồng bộ, tích cực mức thấp. Từ u_reset_sync

clk_en_10khz	Input	1	Clock enable 10 kHz từ u_freq_divider. Enable cho tầng 2 và tầng 3
button_in	Input	1	Tín hiệu từ nút bấm, bất đồng bộ với clk và chưa lọc nhiễu
pulse_out	Output	1	Pulse tích cực mức cao đúng 1 chu kỳ clk tại mỗi rising edge của tín hiệu đã debounce

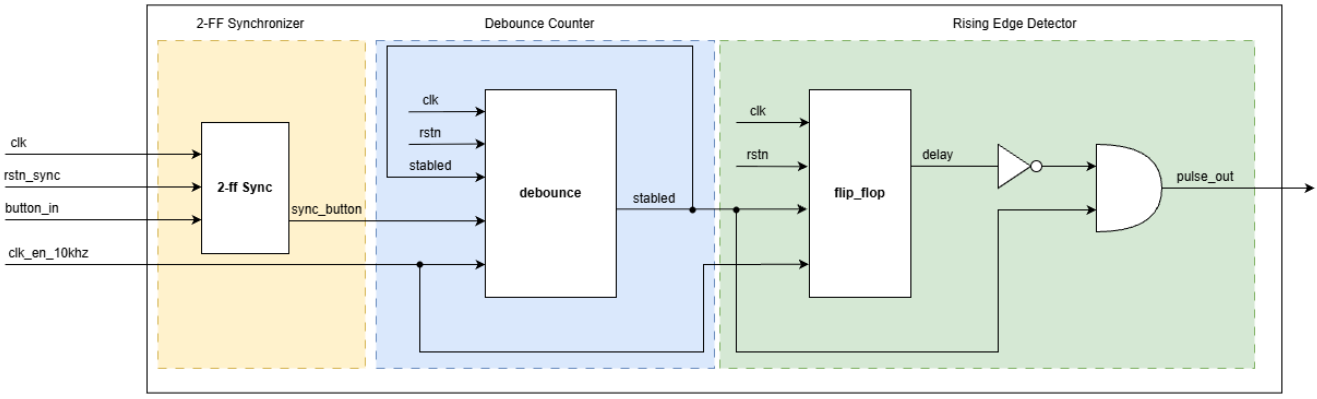
Parameter:

Bảng 3.6: Parameter of module up_down_cond

Parameter	**Type**	**Mặc định**	**Mô tả**
P_DEBOUNCE	integer	100	Số chu kỳ clk_en_10khz liên tục mà input phải giữ ổn định để được công nhận. Mặc định 100 cycles tương ứng 10 ms.

3.2.3. Logic hoạt động chi tiết

Để xử lý tín hiệu đầu vào cho cả 2 instance là button_in từ bất đồng bộ với clock domain chuyển đổi tín hiệu dạng level thành 1 pulse duy nhất mỗi lần nhấn, thông qua pipeline 3 tầng nối tiếp:



Hình 3.3: Mô tả chi tiết logic hoạt động của module up_down_cond

Tầng 1: 2-FF Synchronizer

Internal signals:

Bảng 3.7: Internal signals tầng FF synchronizer module up_down_cond

Signal	Width	Dạng	Mô tả
meta_button	1	reg	Flip-flop tầng 1, có thể bị metastable tại cạnh của button_in nếu gần posedge clk
synced	1	reg	Flip-flop tầng 2, giá trị đã ổn định sau khi FF1 có 1 chu kỳ clock để resolve

Hành vi:

Bảng 3.8: Hành vi signals tầng FF synchronizer module up_down_cond

Điều kiện	Hành vi
rstn_sync = 0	meta_button <= 0, synced <= 0 (async reset)
rstn_sync = 1, posedge clk	meta_button <= button_in, synced <= meta_button

Sử dụng 2 flip-flop mắc nối tiếp. FF1 sample trực tiếp button_in, có xác suất rơi vào metastable. FF2 sample output của FF1 sau 1 chu kỳ clock chờ ổn định. Output synced phản ánh button_in với delay 2 chu kỳ clock, đã an toàn trong clock domain.

Tầng 2: Debounce

Internal signals:

Bảng 3.9: Internals signals tầng debounce module up_down_cond

Signal	Width	Dạng	Mô tả
deb_counter	7	reg	Counter đếm số cycle clk_en_10khz mà input giữ ổn định ở trạng thái khác stabled.
stabled	1	reg	Giá trị input đã được xác nhận ổn định sau debounce. Dạng level, high khi nút đang được nhấn, low khi nhả

Hành vi (chỉ cập nhật khi clk_en_10khz = 1):

Bảng 3.10: Hành vi tầng debounce module up_down_cond

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	stabled <= 0, deb_count <= 0
2	synced != stabled, clk_en_10khz = 1, deb_count == P_DEBOUNCE − 1	stabled <= synced, deb_count <= 0 (accept giá trị mới)
3	synced != stabled, clk_en_10khz = 1, deb_count < P_DEBOUNCE − 1	deb_count <= deb_count + 1 (đếm lên)
4	synced != stabled, clk_en_10khz = 0	deb_count <= deb_count (giữ nguyên counter)
5	synced == stabled	deb_count <= 0 (reset counter — idle hoặc glitch bounce quay về)

Khối debounce so sánh synced với stabled (hồi tiếp từ chính output) mỗi cycle clock clk. Khi 2 giá trị khác nhau, counter đếm lên 1 mỗi cycle clk_en_10khz. Nếu trong quá trình đếm mà synced quay lại bằng stabled (do bounce), counter bị reset về 0 và phải đếm lại từ đầu. Chỉ khi counter đạt P_DEBOUNCE − 1 thì stabled mới được cập nhật bằng synced.

Output stabled dạng level, chuyển trạng thái sau khi input giữ ổn định đủ 10 ms. Mỗi lần chuyển trạng thái, stabled trễ P_DEBOUNCE cycles clk_en_10khz so với synced.

Tầng 3: Rising Edge Detector

Internal signals:

Bảng 3.11: Internal signals tầng rising edge detector module up_down_cond

Signal	Width	Dạng	Mô tả

delay	1	reg	Phiên bản trễ 1 cycle clk_en_10khz của stabled. Dùng để phát hiện rising edge

Hành vi:

Bảng 3.12: Hành vi tầng rising edge detector module up_down_cond

Điều kiện	Hành vi
rstn_sync = 0	delay <= 0
clk_en_10khz = 1	delay <= stabled
clk_en_10khz = 0	Giữ nguyên giá trị cũ

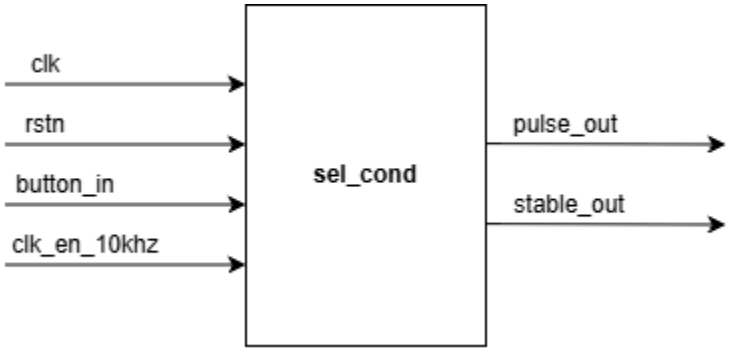
Logic tổ hợp sinh pulse_out:

```
pulse_out = stabled & ~delay
```

Khi stabled chuyển từ 0 lên 1 tại một cycle clk_en_10khz thứ k, flip-flop delay lúc đó vẫn còn giữ giá trị 0 (vì sample giá trị cũ của stabled). Tại chính cycle k+1 (cycle clk_en_10khz kế tiếp), delay mới cập nhật lên 1. Trong khoảng giữa 2 lần clk_en_10khz liên tiếp đó, tổ hợp stabled & ~delay = 1 & ~0 = 1, nên pulse_out = 1 kéo dài đúng 1 chu kỳ clk_en_10khz (100 cycles clk tương ứng 100 µs). Sau đó delay lên 1, pulse_out tự động về 0.

3.3. Block: sel_cond (Select Button Conditioning)

3.3.1. Tổng quan



Hình 3.4: Block diagram module sel_cond

sel_cond là khối xử lý tín hiệu đầu vào từ nút Select. So với nút Up/Down, nút Select cần phân biệt được 2 loại thao tác:

- Short press: trong chế độ chỉnh, chuyển trường thời gian
- Long press (3s): từ NORMAL, vào chế độ chỉnh

Do đó khối có 2 output:

- pulse_out: pulse 1 cycle clk_en_10khz tại rising edge (dùng cho short press)
- stable_out: tín hiệu level, high khi nút đang được giữ (dùng cho long press detect trong u_sel_fsm)

Về cấu trúc, sel_cond dùng lại y hệt pipeline 3 tầng của up_down_cond (2-FF Synchronizer, Debounce Counter, Rising Edge Detector), chỉ thêm 1 đường output stable_out lấy trực tiếp từ stabled của tầng 2.

Được sử dụng 1 lần trong **hms_timer** với instance name: u_sel_cond.

3.3.2. I/O Table

Parameter:

Bảng 3.13: Parameter of module sel_cond

Parameter	Type	Mặc định	Mô tả

P_DEBOUNCE	integer	100	Số chu kỳ clock 10kHz liên tục mà input phải giữ ổn định để được công nhận. Mặc định là 10ms.

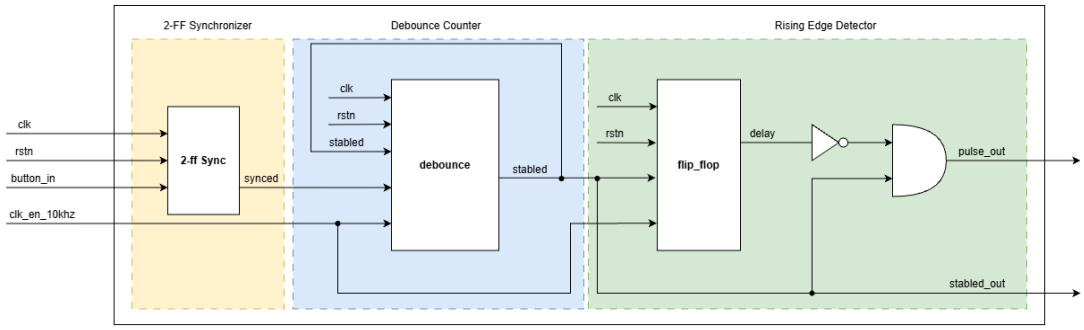
Port:

Bảng 3.14: I/O table module sel_cond

Port	Chiều	Width	Mô tả
clk	Input	1	Clock hệ thống 1 MHz
rstn_sync	Input	1	Reset bất đồng bộ (assert) và đồng bộ (deassert), tích cực mức thấp. Từ u_reset_sync
clk_en_10khz	Input	1	Clock enable 10 kHz từ u_freq_divider. Enable cho tầng 2 và tầng 3
button_in	Input	1	Tín hiệu từ nút Select, bất đồng bộ với clk và chưa lọc nhiễu
pulse_out	Output	1	Pulse tích cực mức cao tại rising edge của tín hiệu đã debounce. Dùng cho chuyển trường
stable_out	Output	1	Tín hiệu level đã debounce. High khi nút đang giữ, low khi nhả. Dùng cho u_sel_fsm để đếm thời gian giữ nút (long press 3s)

3.3.1. Logic hoạt động chi tiết

Hành vi giống với 3 tầng của khối **up_down_cond,** chỉ thêm 1 đường output từ tầng 2 đi ra.



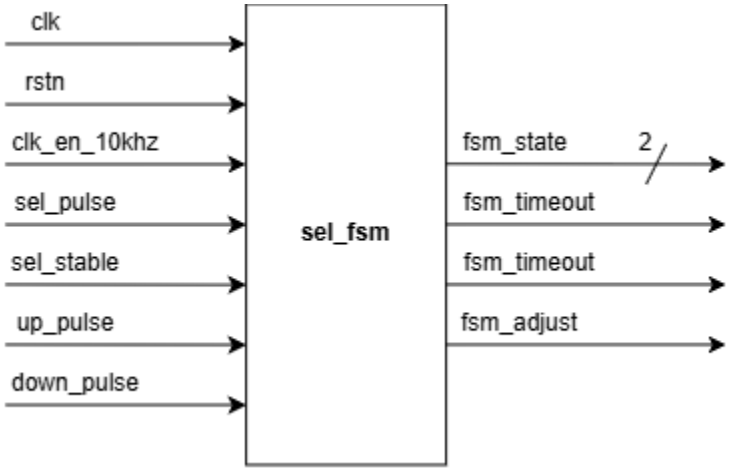
Hình 3.5: Detail block diagram module sel_cond

Điểm khác biệt duy nhất ở output:

Output	Nguồn	Loại	Dùng cho
pulse_out	stabled & ~delay	Pulse (1 cycle clk_en_10khz)	sel_pulse sang u_sel_fsm: chuyển trường trong chế độ chỉnh
stable_out	stabled (lấy trực tiếp từ reg tầng 2)	Level	sel_stable sang u_sel_fsm: đếm cycle để phát hiện long press 3s

3.4. Block: sel_fsm (Select Field FSM)

3.4.1. Tổng quan



Hình 3.6: Block diagram module sel_fsm

sel_fsm là máy trạng thái quản lý chế độ hoạt động của đồng hồ. Gồm 4 trạng thái với 2 cơ chế bảo vệ:

Long press detect: Từ NORMAL, phải giữ nút Select liên tục 3 giây mới vào chế độ chỉnh . Short press ở NORMAL bị bỏ qua.

Timeout: Trong chế độ chỉnh, nếu 5 giây không có bất kỳ thao tác nào (sel, up, down), tự động quay về NORMAL.

Khối phát 3 pulse điều khiển cho u_control_mux:

- fsm_adjust: pulse khi chuyển từ NORMAL sang SEL_MIN (trigger copy main sang adjust)
- fsm_confirm: pulse khi chuyển từ SEL_SEC sang NORMAL qua sel_pulse (trigger load adjust về main)
- fsm_timeout: pulse khi chuyển về NORMAL do timeout (trigger clear adjust_counter)

Khối sử dụng 1 lần trong hms_timer với instance name u_sel_fsm.

3.4.2. I/O Table

Parameter:

Bảng 3.15: Paramenter of module sel_fsm

Parameter	Type	Mặc định	Mô tả
P_LONG_PRESS_THRESHOLD	integer	30_000	Số cycle 10 kHz giữ nút để vào chế độ chỉnh.
P_TIMEOUT_THRESHOLD	integer	50_000	Số cycle 10 kHz không hoạt động trước khi timeout.

Port:

Bảng 3.16: I/O table module sel_fsm

Port	Chiều	Width	Kết nối	Mô tả
clk	In	1	từ top	Clock hệ thống 1 MHz
rstn_sync	In	1	từ u_reset_sync	Reset bất đồng bộ (assert) và đồng bộ (deassert), tích cực mức thấp
clk_en_10khz	In	1	từ u_freq_divider	Clock enable 10 kHz. Mọi logic sequential của FSM chỉ cập nhật khi tín hiệu này active
sel_pulse	In	1	từ u_sel_cond	Short press Select. Chuyển trường trong chế độ chỉnh

sel_stable	In	1	từ u_sel_cond	Level giữ nút Select. Dùng đếm 3 s long press
up_pulse	In	1	từ u_up_cond	Pulse nút Up. Reset counter timeout trong chế độ chỉnh
down_pulse	In	1	từ u_down_cond	Pulse nút Down. Reset counter timeout trong chế độ chỉnh
fsm_state	Out	2	tới u_adjust_counter, u_control_mux	Trạng thái FSM. 00 = NORMAL, 01 = SEL_MIN, 10 = SEL_HOUR, 11 = SEL_SEC
fsm_adjust	Out	1	tới u_control_mux	Pulse khi chuyển NORMAL sang SEL_MIN. Trigger copy main_counter sang adjust_counter
fsm_confirm	Out	1	tới u_control_mux	Pulse khi chuyển SEL_SEC sang NORMAL qua sel_pulse. Trigger load adjust_counter về main_counter
fsm_timeout	Out	1	tới u_control_mux	Pulse khi về NORMAL do timeout. Trigger clear adjust_counter về 0

3.4.3. Logic hoạt động chi tiết

Khối sel_fsm hoạt động khi clk_en_10khz = 1. Và với tín hiệu *activity = sel_pulse | up_pulse | down_pulse.*

State encoding:

Bảng 3.17: State encoding

State	Decode	Mô tả
PL_NORMAL	2'b00	Chế độ đếm bình thường

PL_SEL_MIN	2'b01	Chế độ chỉnh phút
PL_SEL_HOUR	2'b10	Chế độ chỉnh giờ
PL_SEL_SEC	2'b11	Chế độ chỉnh giây

Internal signals:

Bảng 3.18: Internal signals of the module sel_fsm

Signal	Width	Dạng	Mô tả
state	2	reg	Thanh ghi trạng thái FSM hiện tại
inter_counter	16	reg	Counter dùng chung cho long press detect (ở NORMAL) và timeout detect (ở các state chỉnh).
inter_adjust	1	reg	Thanh ghi output pulse fsm_adjust
inter_confirm	1	reg	Thanh ghi output pulse fsm_confirm
inter_timeout	1	reg	Thanh ghi output pulse fsm_timeout
activity	1	wire	Tổ hợp sel_pulse | up_pulse | down_pulse. Mọi hoạt động nút reset counter timeout

Chia sẻ counter cho 2 chức năng:

Khối dùng chung 1 counter inter_counter vì long press detect chỉ hoạt động ở PL_NORMAL, timeout chỉ hoạt động ngoài PL_NORMAL. Hai chức năng không bao giờ chạy đồng thời.

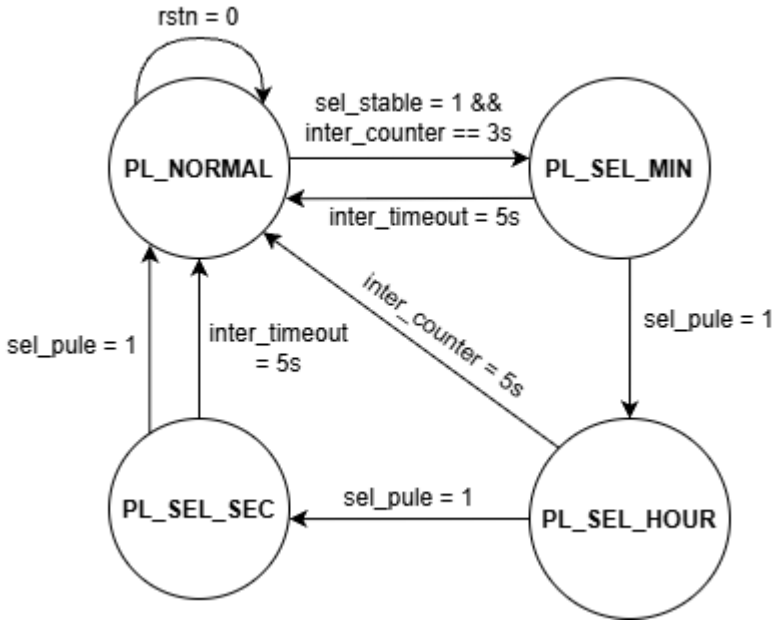
Bảng 3.19: Share function logic counter

State	Vai trò counter	Threshold	Khi đạt threshold

PL_NORMAL	Long press detect	P_LONG_PRESS_THRESHOLD (3 s)	Chuyển sang PL_SEL_MIN
PL_SEL_MIN / PL_SEL_HOUR / PL_SEL_SEC	Timeout	P_TIMEOUT_THRESHOLD (5 s)	Chuyển về PL_NORMAL

Counter tự reset về 0 khi chuyển trạng thái, sẵn sàng đếm cho chức năng mới.

State diagram:



Hình 3.7: State diagram cho hoạt động của module sel_fsm

State: PL_NORMAL

Trạng thái mặc định sau reset, đồng hồ đếm bình thường. Bộ counter phục vụ cho đếm long press detect với THESHOLD = 30000.

Bảng 3.20: Điều kiện ưu tiên và hành vi tại state PL_NORMAL

Priority	**Điều kiện**	**Hành vi**

1	rstn_sync = 0	state <= PL_NORMAL, inter_counter <= 0
2	sel_stable = 0	inter_counter <= 0 (nút chưa nhấn hoặc đã nhả, reset counter)
3	sel_stable = 1, inter_counter == P_LONG_PRESS_THRESHOLD − 1 (29999)	state <= PL_SEL_MIN, inter_counter <= 0 (long press detected)
4	sel_stable = 1, inter_counter < 29_999	inter_counter <= inter_counter + 1 (đang đếm long press)

State: PL_SEL_MIN

Trạng thái báo hiệu vào chế độ chỉnh phút, khối counter phục vụ việc đếm timeout với THRESHOLD = 50000 (5s).

Bảng 3.21: Điều kiện ưu tiên và hành vi tại state PL_SEL_MIN

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	state <= PL_NORMAL, inter_counter <= 0
2	sel_pulse = 1	state <= PL_SEL_HOUR, inter_counter <= 0 (short press, sang trường kế)

3	activity = 1 (không phải sel_pulse, tức là up_pulse hoặc down_pulse)	inter_counter <= 0 (có hoạt động, reset timeout)
4	activity = 0, inter_counter == P_TIMEOUT_THRESHOLD − 1 = 49_999	state <= PL_NORMAL, inter_counter <= 0 (timeout)
5	activity = 0, inter_counter < 49_999	inter_counter <= inter_counter + 1 (đang đếm timeout)

Khi chuyển về trạng thái PL_NORMAL thì đồng thời tín hiệu fsm_timeout = 1.

State: PL_SEL_HOUR

Trạng thái chỉnh giờ, bộ counter tiếp tục vai trò đếm timeout với THRESHOLD = 50000 (5s).

Bảng 3.22: Điều kiện ưu tiên và hành vi tại state PL_SEL_HOUR

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	state <= PL_NORMAL, inter_counter <= 0
2	sel_pulse = 1	state <= PL_SEL_SEC, inter_counter <= 0 (short press, sang trường kế)
3	activity = 1 (không phải sel_pulse, tức là up_pulse hoặc down_pulse)	inter_counter <= 0 (có hoạt động, reset timeout)

4	activity = 0, inter_counter == P_TIMEOUT_THRESHOLD − 1 = 49_999	state <= PL_NORMAL, inter_counter <= 0 (timeout)
5	activity = 0, inter_counter < 49_999	inter_counter <= inter_counter + 1 (đang đếm timeout)

Khi chuyển về trạng thái PL_NORMAL thì đồng thời tín hiệu fsm_timeout = 1.

State: PL_SEL_SEC

Trạng thái chỉnh giây, bộ counter tiếp tục vai trò đếm timeout với THRESHOLD = 50000 (5s).

Bảng 3.23: Điều kiện ưu tiên và hành vi tại state PL_SEL_SEC

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	state <= PL_NORMAL, inter_counter <= 0
2	sel_pulse = 1	state <= PL_NORMAL, inter_counter <= 0 (short press, sang trường kế)
3	activity = 1 (không phải sel_pulse, tức là up_pulse hoặc down_pulse)	inter_counter <= 0 (có hoạt động, reset timeout)
4	activity = 0, inter_counter == P_TIMEOUT_THRESHOLD − 1 = 49_999	state <= PL_NORMAL, inter_counter <= 0 (timeout)
5	activity = 0, inter_counter < 49_999	inter_counter <= inter_counter + 1

		(đang đếm timeout)

Khi chuyển về trạng thái PL_NORMAL với sel_pulse == 1 thì output ra khối là inter_confirm = 1. Còn khi chuyển về trạng thái NORMAL với inter_counter == 49999 (điều kiện timeout) thì output ra khỏi khối là fsm_timeout = 1.

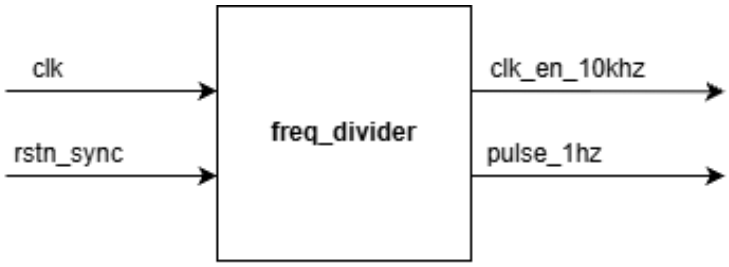
Case default và điều kiện ngoài clk_en_10khz:

Bảng 3.24: Case default và điều kiện ngoài clk_en_10khz

Priority	Điều kiện	Hành vi
default	state không hợp lệ	state <= PL_NORMAL, inter_counter <= 0
hold	clk_en_10khz = 0	state <= state, inter_counter <= inter_counter (giữ nguyên)

3.5. Block: freq_divider (Frequency Divider)

3.5.1. Tổng quan



Hình 3.8: Block diagram module freq_divider

freq_divider là bộ chia tần số, chuyển clock hệ thống 1 MHz thành 2 tín hiệu enable cho các khối phía sau.

- *clk_en_10khz*: pulse 1 cycle clk xuất hiện mỗi 100 µs (tần số 10 kHz). Dùng làm clock enable cho các khối sel_cond, up_down_cond, sel_fsm, main_counter, adjust_counter, control_mux
- *pulse_1hz*: pulse 1 cycle clk xuất hiện mỗi 1 s (tần số 1 Hz). Dùng làm trigger cho main_counter tăng giây.

Được sử dụng 1 lần trong hms_timer với instance name: u_freq_divider.

3.5.2. I/O Table

Port:

Bảng 3.25: I/O table module freq_divider

Port	Chiều	Width	Kết nối	Mô tả
clk	In	1	Từ top	Clock hệ thống 1 MHz
rstn_sync	In	1	Từ u_reset_sync	Reset bất đồng bộ và đồng bộ, tích cực mức thấp
clk_en_10khz	Out	1	Tới u_sel_cond, u_up_cond, u_down_cond, u_sel_fsm, u_main_counter, u_adjust_counter, u_control_mux	Pulse 1 cycle clk mỗi 100 µs
pulse_1hz	Out	1	Tới u_main_counter	Pulse 1 cycle clk mỗi 1 s

3.5.3. Logic hoạt động chi tiết

Internal signal

Bảng 3.26: Internal signals of the module freq_divider

Signal	Width	Dạng	Mô tả
count_10khz	7	reg	Counter đếm từ 0 tới 99. Khi đạt 99, wrap về 0 và sinh pulse inter_10khz.

inter_10khz	1	reg	Thanh ghi output pulse clk_en_10khz. High đúng 1 cycle clk khi count_10khz wrap
count_1hz	14	reg	Counter đếm từ 0 tới 9999, chỉ tăng khi count_10khz >= 99. Khi đạt 9999, wrap về 0 và sinh pulse inter_1hz.
inter_1hz	1	reg	Thanh ghi output pulse pulse_1hz. High đúng 1 cycle clk khi count_1hz wrap đồng thời với count_10khz wrap

Module **freq_divider** gồm 2 khối riêng biệt để gen các tín hiệu inter_10khz và inter_1hz:

Block 1: clk_en_10khz generation

Bảng 3.27: Mức độ ưu tiên giữa các điều kiện và hành vi tương ứng của logic clk_en_10khz generation

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	count_10khz <= 0, inter_10khz <= 0
2	count_10khz >= 99	count_10khz <= 0, inter_10khz <= 1 (wrap và phát pulse)
3	count_10khz < 99	count_10khz <= count_10khz + 1, inter_10khz <= 0 (đếm bình thường)

Counter_10khz chạy liên tục ở 1 MHz, mỗi cycle clk tăng 1. Khi đếm đến 99 (chu kỳ thứ 100 kể từ 0), nó wrap về 0 đồng thời set inter_10khz = 1 trong đúng 1 cycle.

Block 2: pulse_1hz generation

Bảng 3.28: Mức độ ưu tiên giữa các điều kiện và hành vi tương ứng của logic pulse_1hz generation

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	count_1hz <= 0, inter_1hz <= 0
2	count_10khz >= 99, count_1hz >= 9999	count_1hz <= 0, inter_1hz <= 1 (wrap và phát pulse)

3	count_10khz >= 99, count_1hz < 9999	count_1hz <= count_1hz + 1, inter_1hz <= 0 (đếm bình thường)
4	count_10khz < 99	count_1hz giữ nguyên (implicit hold), inter_1hz <= 0

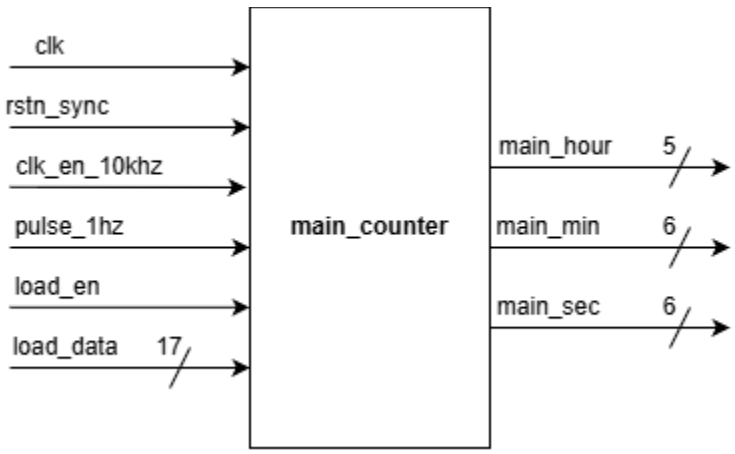
count_1hz chạy mỗi posedge clk nhưng chỉ tăng khi count_10khz >= 99 (cùng cycle inter_10khz = 1). Enable này, count_1hz chỉ tăng 1 lần mỗi 100 µs. Khi đếm đến 9999 (chu kỳ thứ 10 000 của count_1hz) đồng thời có count_10khz >= 99, cả 2 pulse inter_1hz và inter_10khz cùng được set lên 1 trong cùng cycle clk.

Outputs

Clk_en_10khz = inter_10khz Pulse_1hz = inter_1hz

3.6. Block: main_counter (Main Time Counter)

3.6.1. Tổng quan



Hình 3.9: Block diagram module main_counter

main_counter là bộ đếm thời gian chính của đồng hồ. Khối này phụ trách việc đếm và luôn chạy free-run, không bao giờ dừng, không nhận adjust từ nút bấm.

Chứa 3 thanh ghi nội bộ cho sec (0–59), min (0–59), hour (0–23) với carry chain: sec tràn 59 → carry lên min, min tràn 59 → carry lên hour, hour wrap tại 23 về 0.

Khối hỗ trợ load dữ liệu từ bên ngoài qua load_en và load_data khi user hoàn thành chỉnh giờ (confirm), control_mux nạp giá trị đã chỉnh từ adjust_counter vào main_counter, đồng hồ đếm tiếp từ giá trị mới.

Sử dụng clk_en_10khz làm clock enable. Được sử dụng 1 lần trong hms_timer với instance name: u_main_counter.

3.6.2. I/O Table

Bảng 3.29: I/O table module main_counter

Port	Chiều	Width	Kết nối	Mô tả
clk	In	1	từ top	Clock hệ thống 1 MHz
rstn_sync	In	1	từ u_reset_sync	Reset bất đồng bộ và đồng bộ , tích cực mức thấp
clk_en_10khz	In	1	từ u_freq_divider	Clock enable 10 kHz. Mọi logic sequential chỉ cập nhật khi tín hiệu này active
pulse_1hz	In	1	từ u_freq_divider	Pulse 1 Hz. Trigger tăng sec mỗi 1 giây
load_en	In	1	từ u_control_mux	Pulse 1 cycle clk_en_10khz. Cho phép nạp dữ liệu từ load_data vào 3 thanh ghi (ưu tiên trên counting)
load_data	In	17	từ u_control_mux	Dữ liệu nạp từ adjust_counter: {hour[4:0], min[5:0], sec[5:0]}
main_hour	Out	5	tới u_control_mux	Giá trị giờ hiện tại (0–23)
main_min	Out	6	tới u_control_mux	Giá trị phút hiện tại (0–59)
main_sec	Out	6	tới u_control_mux	Giá trị giây hiện tại (0–59)

3.6.3. Logic hoạt động chi tiết

Internal signals:

Bảng 3.30: Internal signals of the module time_counter

Signal	Width	Mô tả
sec	6	Thanh ghi giây (0–59)
min	6	Thanh ghi phút (0–59)
hour	5	Thanh ghi giờ (0–23)
sec_carry	1	Carry từ sec lên min khi sec đạt 59 và có pulse_1hz
min_carry	1	Carry từ min lên hour khi min đạt 59 và có sec_carry

Load data bit mapping:

Bảng 3.31: Load data bit mapping

load_data	Field	Width
load_data[16:12]	hour	5 bits
load_data[11:6]	min	6 bits
load_data[5:0]	sec	6 bits

Toàn bộ logic của khối chạy khi có clk_en_10khz = 1. Cấu trúc gồm 3 khối độc lập về hành vi:

Block SEC

Bảng 3.32: Mức độ ưu tiên điều kiện và hành vi tương ứng của logic update sec register

Priority	Điều kiện	Hành vi

1	rstn_sync = 0	sec <= 0
2	clk_en_10khz = 1, load_en = 1	sec <= load_data[5:0] (load confirm)
3	clk_en_10khz = 1, load_en = 0, pulse_1hz = 1, sec >= 59	sec <= 0 (wrap)
4	clk_en_10khz = 1, load_en = 0, pulse_1hz = 1, sec < 59	sec <= sec + 1 (count up)
5	Các điều kiện còn lại	sec giữ nguyên

SEC block xuất ra tín hiệu sec_carry là kết hợp của pulse_1hz và thanh ghi sec khi thanh ghi này >= 59 (sec_carry = pulse_1hz & (sec >= 59)). sec_carry là cờ carry chain của thanh ghi sec sang min.

Block MIN

Bảng 3.33: Mức độ ưu tiên điều kiện và hành vi tương ứng của logic update min register

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	min <= 0
2	clk_en_10khz = 1, load_en = 1	min <= load_data[11:6] (load confirm)
3	clk_en_10khz = 1, load_en = 0, sec_carry = 1, min >= 59	min <= 0 (wrap)
4	clk_en_10khz = 1, load_en = 0, sec_carry = 1, min < 59	min <= min + 1 (count up)
5	Các điều kiện còn lại	min giữ nguyên

SEC block xuất ra tín hiệu min_carry là kết hợp của sec_carry và thanh ghi min khi thanh ghi này >= 59 (min_carry = sec_carry & (min >= 59)). min_carry là cờ carry chain của thanh ghi min sang hour.

Block HOUR

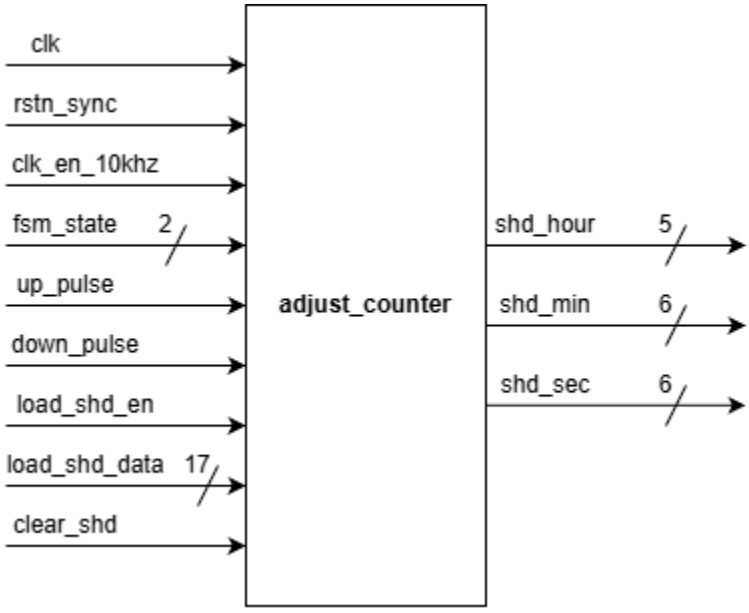
Bảng 3.34: Mức độ ưu tiên điều kiện và hành vi tương ứng của logic update hour register

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	hour <= 0
2	clk_en_10khz = 1, load_en = 1	hour <= load_data[16:12] (load confirm)
3	clk_en_10khz = 1, load_en = 0, min_carry = 1, hour >= 23	hour <= 0 (wrap)
4	clk_en_10khz = 1, load_en = 0, min_carry = 1, hour < 23	hour <= hour + 1 (count up)
5	Các điều kiện còn lại	hour giữ nguyên

Lưu ý về priority Load > Count: Ở cả 3 block, load_en được check trước carry/pulse. Nếu load_en và pulse count fire cùng cycle, load thắng thì phải đảm bảo confirm load đúng giá trị user đã chỉnh.

3.7. Block: adjust_counter (Adjust Counter)

3.7.1. Tổng quan



Hình 3.10: Block diagram module adjust_counter

adjust_counter là bộ thanh ghi phục vụ chế độ chỉnh thời gian. Khối này chỉ hoạt động khi đồng hồ ở chế độ chỉnh (fsm_state ≠ PL_NORMAL), chạy song song với main_counter nhưng không nhận pulse_1hz (không tự đếm).

Khối thực hiện 3 việc chính:

- *Load:* Khi vào chế độ chỉnh (từ fsm_adjust), nhận giá trị copy từ main_counter qua load_shd_data làm giá trị khởi tạo cho user chỉnh
- *Adjust:* Nhận up_pulse và down_pulse, tăng/giảm giá trị trường đang chọn (theo **fsm_state**). Giá trị chỉ wrap trong trường đang chỉnh, không carry/borrow sang trường khác.
    - *Clear:* Khi clear_shd = 1 (từ fsm_timeout), reset cả 3 reg về 0.

Khi fsm_confirm, control_mux lấy output của adjust_counter nạp vào main_counter. Khi fsm_timeout, control_mux phát clear_shd, shadow về 0 và không hiển thị nữa (main_counter tiếp tục hiển thị).

Khối sử dụng clk_en_10khz làm clock enable. Instance name: u_adjust_counter.

3.7.2. I/O Table

Bảng 3.35: I/O table of the module adjust_counter

Port	Chiều	Width	Kết nối	Mô tả
clk	In	1	từ top	Clock hệ thống 1 MHz
rstn_sync	In	1	từ u_reset_sync	Reset bất đồng bộ và đồng bộ, tích cực mức thấp
clk_en_10khz	In	1	từ u_freq_divider	Clock enable 10 kHz
fsm_state	In	2	từ u_sel_fsm	Trường đang chỉnh:MIN, HOUR, SEC. Xác định Up/Down tác động trường nào
up_pulse	In	1	từ u_up_cond	Tăng giá trị trường đang chọn. Chỉ có tác dụng khi fsm_state ≠ PL_NORMAL
down_pulse	In	1	từ u_down_cond	Giảm giá trị trường đang chọn. Chỉ có tác dụng khi fsm_state ≠ PL_NORMAL
load_shd_en	In	1	từ u_control_mux	Pulse 1 cycle clk_en_10khz. Cho phép nạp dữ liệu từ load_shd_data vào 3 reg (vào chế độ chỉnh)
load_shd_data	In	17	từ u_control_mux	Dữ liệu nạp từ main_counter: {hour[4:0], min[5:0], sec[5:0]}
clear_shd	In	1	từ u_control_mux	Pulse 1 cycle clk_en_10khz. Clear cả 3 reg về 0 khi timeout
shd_hour	Out	5	tới u_control_mux	Giá trị giờ shadow (0–23)
shd_min	Out	6	tới u_control_mux	Giá trị phút shadow (0–59)
shd_sec	Out	6	tới u_control_mux	Giá trị giây shadow (0–59)

3.7.3. Logic hoạt động chi tiết

Internal signals:

Bảng 3.36: Internal signals of the module adjust_counter

Signal	Width	Dạng	Mô tả
sec	6	reg	Thanh ghi giây (0–59).
min	6	reg	Thanh ghi phút (0–59).
hour	5	reg	Thanh ghi giờ (0–23).
action	2	wire	Hành động chung từ Up/Down.
sec_adj	2	wire	Hành động cho sec sau khi route theo fsm_state. Chỉ ≠ 00 khi fsm_state = SEL_SEC
min_adj	2	wire	Hành động cho min. Chỉ ≠ 00 khi fsm_state = SEL_MIN
hour_adj	2	wire	Hành động cho hour. Chỉ ≠ 00 khi fsm_state = SEL_HOUR

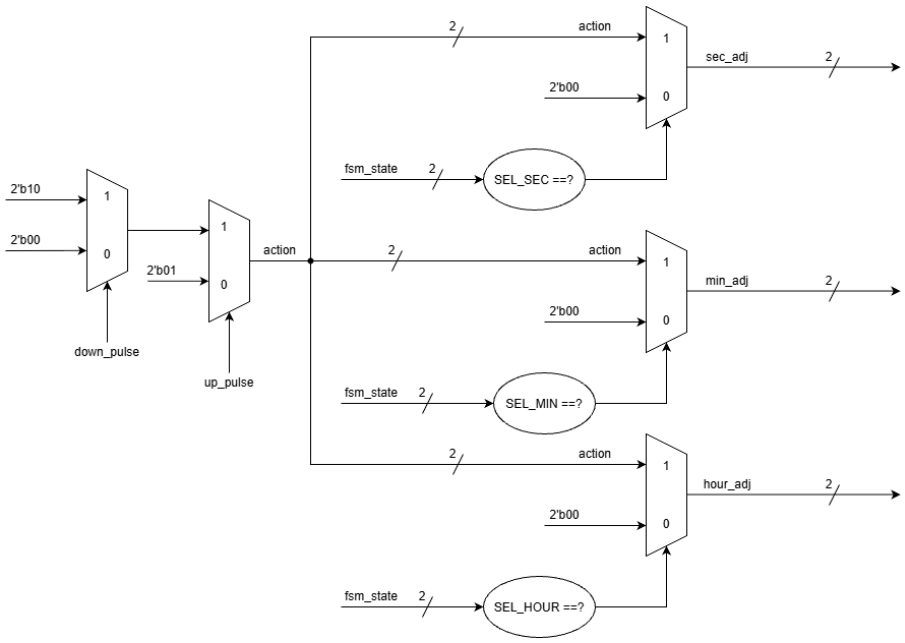
Localparams (state encoding cho FSM decoder)

Bảng 3.37: State encoding for FSM decoder

Localparam	Mã	Mô tả
PL_NORMAL	2'b00	Chế độ bình thường, không có adjust
PL_SEL_MIN	2'b01	Chỉnh phút — route action sang min_adj
PL_SEL_HOUR	2'b10	Chỉnh giờ — route action sang hour_adj
PL_SEL_SEC	2'b11	Chỉnh giây — route action sang sec_adj

Cấu trúc khối gồm 2 phần:

Phần 1: FSM decoder



Hình 3.11: Detail logic of FSM decoder

Logic hoạt động của FSM decoder được mô tả theo **Hình 3.11** gồm 2 bước:

Tính action từ Up/Down (ưu tiên Up):

Bảng 3.38: Logic cập nhật giá trị action

up_pulse	down_pulse	action
0	0	00 (hold)
1	0	01 (+1)
1	1	01 (+1), (Up ưu tiên)
0	1	10 (−1)

Route action sang trường tương ứng theo fsm_state:

Bảng 3.39: Route action sang trường tương ứng theo fsm_state

fsm_state	sec_adj	min_adj	hour_adj

PL_NORMAL (00)	00	00	00
PL_SEL_MIN (01)	00	action	00
PL_SEL_HOUR (10)	00	00	action
PL_SEL_SEC (11)	action	00	00

Phần 2: Registers – update logic

SEC register

Bảng 3.40: Mức độ ưu tiên của điều kiện và hành vi tương ứng của cập nhật logic cho sec register

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	sec <= 0
2	clk_en_10khz = 1, clear_shd = 1	sec <= 0 (clear khi timeout)
3	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 1	sec <= load_shd_data[5:0] (load từ main)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, sec_adj = 2'b01	sec <= (sec >= 59) ? 0 : sec + 1 (Up wrap)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, sec_adj = 2'b10	sec <= (sec == 0) ? 59 : sec - 1 (Down wrap)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, sec_adj = 2'b00 (default)	sec <= sec (hold)
5	clk_en_10khz = 0	sec giữ nguyên

MIN register

Bảng 3.41: Mức độ ưu tiên của điều kiện và hành vi tương ứng của cập nhật logic cho min register

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	min <= 0
2	clk_en_10khz = 1, clear_shd = 1	min <= 0 (clear khi timeout)
3	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 1	min <= load_shd_data[11:6] (load từ main)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, min_adj = 2'b01	min <= (min >= 59) ? 0 : min + 1 (Up wrap)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, min_adj = 2'b10	min <= (min == 0) ? 59 : min - 1 (Down wrap)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, min _adj = 2'b00	min <= min (hold)
5	clk_en_10khz = 0	min giữ nguyên

HOUR register

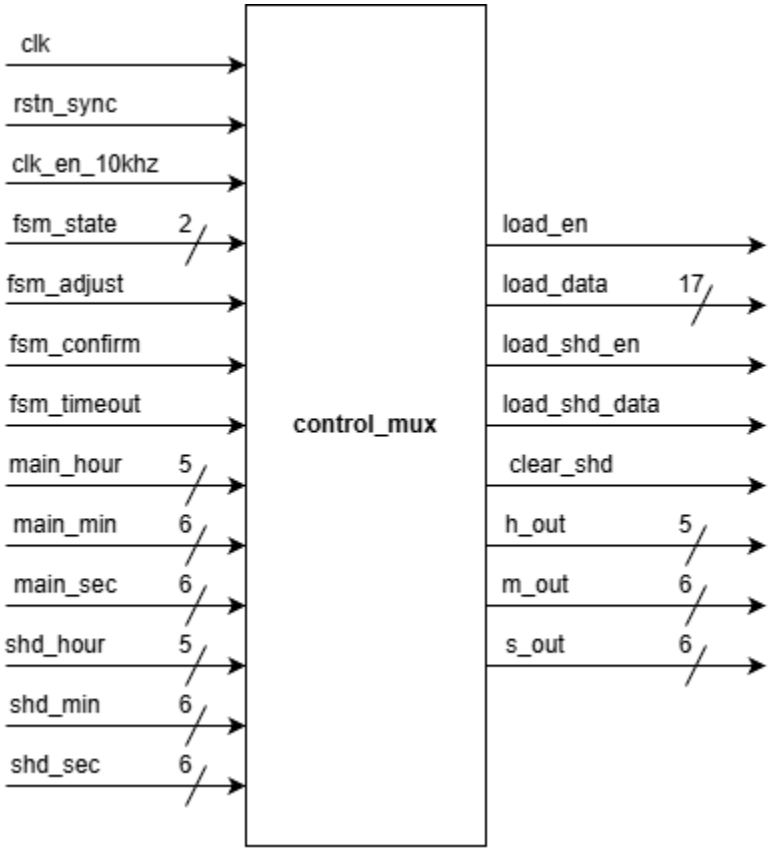
Bảng 3.42: Mức độ ưu tiên của điều kiện và hành vi tương ứng của cập nhật logic cho hour register

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	hour <= 0
2	clk_en_10khz = 1, clear_shd = 1	hour <= 0 (clear khi timeout)
3	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 1	hour <= load_shd_data[16:12] (load từ main)

4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, hour_adj = 2'b01	hour <= (hour >= 23) ? 0 : hour + 1 (Up wrap)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, hour_adj = 2'b10	hour <= (hour == 0) ? 23 : hour - 1 (Down wrap)
4	clk_en_10khz = 1, clear_shd = 0, load_shd_en = 0, hour _adj = 2'b00	hour <= hour (hold)
5	clk_en_10khz = 0	hour giữ nguyên

3.8. Block: control_mux (Control and Output MUX)

3.8.1. Tổng quan



Hình 3.12: Block diagram module control_mux

control_mux là khối điều phối của hms_timer. Khối thực hiện 3 chức năng:

Điều phối load: Phát load_shd_en khi vào chế độ chỉnh (copy dữ liệu từ main sang adjust), phát load_en khi confirm (load dữ liệu shadow quay trở lại main).

Đóng gói dữ liệu: Gộp giá trị {hour, min, sec} từ main_counter thành load_shd_data 17-bit, và từ adjust_counter thành load_data 17-bit.

Chọn hiển thị: Dựa vào fsm_state để quyết định hiển thị main hay adjust ra output.

Khối kết hợp giữa logic tổ hợp (load, đóng gói) và sequential (thanh ghi fsm_state_d cho MUX hiển thị) để đảm bảo output. Được sử dụng 1 lần trong hms_timer với instance name: u_control_mux.

3.8.2. I/O Table

Bảng 3.43: I/O table of the control_mux module

Port	Chiều	Width	Kết nối	Mô tả
clk	In	1	từ top	Clock hệ thống 1 MHz
rstn_sync	In	1	từ u_reset_sync	Reset bất đồng bộ và đồng bộ, tích cực mức thấp
clk_en_10khz	In	1	từ u_freq_divider	Clock enable 10 kHz. Chỉ dùng cho sequential update của fsm_state_d
fsm_state	In	2	từ u_sel_fsm	Trạng thái FSM. Dùng cho sample vào fsm_state_d
fsm_adjust	In	1	từ u_sel_fsm	Pulse khi PL_NORMAL chuyển sang PL_SEL_MIN. Trigger copy main sang adjust
fsm_confirm	In	1	từ u_sel_fsm	Pulse khi PL_SEL_SEC chuyển sang PL_NORMAL qua sel_pulse. Trigger load adjust về main
fsm_timeout	In	1	từ u_sel_fsm	Pulse khi về PL_NORMAL do timeout. Trigger clear adjust về 0
main_hour	In	5	từ u_main_counter	Giá trị giờ bộ chính
main_min	In	6	từ u_main_counter	Giá trị phút bộ chính
main_sec	In	6	từ u_main_counter	Giá trị giây bộ chính
shd_hour	In	5	từ u_adjust_counter	Giá trị giờ đã chỉnh
shd_min	In	6	từ u_adjust_counter	Giá trị phút đã chỉnh
shd_sec	In	6	từ u_adjust_counter	Giá trị giây đã chỉnh

load_en	Out	1	tới u_main_counter	Enable load main. Pulse 1 cycle clk_en_10khz khi confirm
load_data	Out	17	tới u_main_counter	Dữ liệu nạp main = {shd_hour, shd_min, shd_sec}
load_shd_en	Out	1	tới u_adjust_counter	Enable load adjust. Pulse 1 cycle clk_en_10khz khi vào chế độ chỉnh
load_shd_data	Out	17	tới u_adjust_counter	Dữ liệu nạp adjust = {main_hour, main_min, main_sec}
clear_shd	Out	1	tới u_adjust_counter	Clear adjust về 0. Pulse 1 cycle clk_en_10khz khi timeout
h_out	Out	5	tới top	Giờ hiển thị: fsm_state_d = PL_NORMAL → main, khác → shadow
m_out	Out	6	tới top	Phút hiển thị: tương tự h_out
s_out	Out	6	tới top	Giây hiển thị: tương tự h_out

3.8.3. Logic hoạt động chi tiết

Internal signals:

Bảng 3.44: Internal signal of the control_mux module

Signal	Width	Dạng	Mô tả
fsm_state_d	2	Reg	Phiên bản trễ 1 cycle của fsm_state. Dùng để chọn MUX hiển thị, đảm bảo dữ liệu đích (main hoặc shadow) đã load xong trước khi MUX đổi hiển thị → không có glitch

Localparam:

Bảng 3.45: Localparam of the control_mux module

Localparam	Mã	Mô tả
PL_NORMAL	2'b00	State encoding chế độ bình thường, dùng cho MUX condition

Khối gồm 3 phần logic:

Phần 1: Thanh ghi trễ fsm_state_d

Thanh ghi 2-bit trễ 1 cycle của fsm_state. Hoạt động theo cạnh lên clock với enable clk_en_10khz.

Bảng 3.46: Mức độ ưu tiên về điều kiện và hành vi tương ứng của fsm_sate_d register

Priority	Điều kiện	Hành vi
1	rstn_sync = 0	fsm_state_d <= PL_NORMAL
2	clk_en_10khz = 1	fsm_state_d <= fsm_state (sample)
3	clk_en_10khz = 0	fsm_state_d giữ nguyên (implicit hold)

Phần 2: Điều phối Load

Khối phát 2 cặp tín hiệu load — một cho adjust_counter, một cho main_counter. Cả 2 đều lấy trực tiếp từ pulse của sel_fsm

Load adjust — khi vào chế độ chỉnh

Khi sel_fsm phát fsm_adjust = 1 (PL_NORMAL chuyển sang PL_SEL_MIN), control_mux lập tức phát load_shd_en = 1 để adjust_counter copy giá trị từ main_counter.

Bảng 3.47: Logic phát tín hiệu load_shd_en và đích đến của nhóm bit load_shd_data

Tín hiệu	Giá trị	Mô tả
load_shd_en	= fsm_adjust	Pulse 1 cycle
load_shd_data[16:12]	= main_hour	Giờ
load_shd_data[11:6]	= main_min	Phút
load_shd_data[5:0]	= main_sec	Giây

load_shd_data là giá trị tổ hợp từ main hiện tại, nhưng chỉ được adjust_counter lấy khi load_shd_en = 1.

Load main — khi confirm

Khi sel_fsm phát fsm_confirm = 1 (PL_SEL_SEC sang PL_ NORMAL), control_mux lập tức phát load_en = 1 để main_counter nhận giá trị đã chỉnh từ shadow_counter.

Bảng 3.48: Logic phát load_en và đích đến của nhóm bit load_data vào main khi confirm

Tín hiệu	Giá trị	Mô tả
load_en	= fsm_confirm	Pulse 1 cycle
load_data[16:12]	= shd_hour	Giờ đã chỉnh
load_data[11:6]	= shd_min	Phút đã chỉnh
load_data[5:0]	= shd_sec	Giây đã chỉnh

Clear adjust – khi timeout

Bảng 3.49: Logic tín hiệu clear_shd

Tín hiệu	Giá trị	Mô tả
clear_shd	= fsm_timeout	Pulse 1 cycle clk_en_10khz khi về PL_NORMAL do timeout

Phần 3: MUX chọn hiển thị

Dựa vào fsm_state_d (trễ 1 cycle của fsm_state) để chọn giữa 2 nguồn dữ liệu.

Bảng 3.50: MUX chọn hiển thị

fsm_state_d	Trạng thái	h_out	m_out	s_out	Giải thích
00 (PL_NORMAL)	Đồng hồ đếm bình thường	main_hour	main_min	main_sec	Hiển thị bộ main
01 (PL_SEL_MIN)	Chỉnh phút	shd_hour	shd_min	shd_sec	Hiển thị bộ adjust (user chỉnh)
10 (PL_SEL_HOUR)	Chỉnh giờ	shd_hour	shd_min	shd_sec	Hiển thị bộ adjust
11 (PL_SEL_SEC)	Chỉnh giây	shd_hour	shd_min	shd_sec	Hiển thị bộ adjust